

	MANUAL TECNICO		
Universidad Piloto de Colombia	PROYECTO: DieselMaps	Grupo: 2	
		Ciclo:	2

MANUAL TECNICO

Version	2.0.0
Fecha	Mayo 2026
Java / Spring Boot	17 LTS / 3.5.5
Frontend	React 19.2 + Vite 8
Base de datos	MySQL 8.x + Flyway
Clasificacion	Uso tecnico interno

Referencia tecnica para desarrolladores, arquitectos y equipo DevOps de DieselMaps

1. introducción

1.1 propósito del documento

Este manual técnico describe la arquitectura, estructura interna, estándares de desarrollo, configuración, despliegue y mantenimiento del sistema DieselMaps v1.0. Sirve como referencia definitiva para el equipo de desarrollo, arquitectos de software, integradores de sistemas y personal de operaciones que trabaje con la plataforma.

1.2 Alcance

- Backend Spring Boot 3.5 (Java 17): capas de controller, service, repository, security y configuration.
- Frontend React 19 + Vite: estructura de componentes, gestión de estado, enrutamiento y cliente HTTP.
- Base de datos MySQL 8.x: modelo relacional, migraciones Flyway y consultas críticas.

DieselMaps, extraído de universidad piloto de Colombia

Firmado por Líder de calidad

Ver 2.0

	MANUAL TECNICO		
Universidad Piloto de Colombia	PROYECTO: DieselMaps	Grupo: 2	
		Ciclo:	2

- Seguridad: flujo completo de autenticación JWT y autorización RBAC.
- Despliegue, mantenimiento y actualización del sistema en entornos productivos.

1.3 Público objetivo

- **Desarrolladores backend/frontend:** para incorporarse al proyecto y extender funcionalidades.
- **Arquitectos de software:** para evaluar el diseño y proponer mejoras tecnológicas.
- **Ingenieros DevOps:** para configurar CI/CD, gestionar entornos y automatizar despliegues.
- **Audidores de seguridad:** para revisar el modelo de autenticación y manejo de datos sensibles.

2. descripción General del Sistema

2.1 propósito

DieselMaps es una plataforma web para la gestión, consulta y compra virtual de combustible en estaciones de servicio colombianas. Integra cartografía interactiva, gestión de inventario en tiempo real, historial de precios, favoritos con alertas automáticas y una tienda virtual con transacciones atómicas sobre el stock.

2.2 Arquitectura de alto nivel

El sistema implementa una arquitectura cliente-servidor de dos capas estrictamente desacopladas. El backend es completamente stateless gracias al uso de JWT, lo que facilita la escalabilidad horizontal sin necesidad de coordinación de sesiones entre instancias.

[React 19 SPA] [Spring Boot 3.5 API] [MySQL 8.x]
 Puerto 5173 --HTTP--> Puerto 8080 --JDBC--> Puerto 3306
 React Router Spring Security + JWT Flyway

DieselMaps, extraído de universidad piloto de Colombia

Firmado por líder de calidad

Ver 2.0

	MANUAL TECNICO	
Universidad Piloto de Colombia	PROYECTO: DieselMaps	Grupo: 2
		Ciclo: 2

Axios + interceptors Spring Data JPA InnoDB / utf8mb4
 AuthContext + localStorage BCrypt + RBAC 11 tablas

Decisiones de arquitectura:

- Stateless (JWT): permite replicación horizontal sin sticky sessions
- separación estricta backend/frontend: deploy independiente
- Flyway: trazabilidad de cambios de esquema con checksum
- @Transactional READ_COMMITTED: evita race conditions en stock

2.3 tecnologías utilizadas

Capa	Tecnologia	Version	Proposito
Backend	Spring Boot	3.5.5	Framework principal
Backend	Java	17 LTS	Lenguaje de programación
Backend	Spring Security	6.x	autenticación y autorización
Backend	JJWT (jjwt-api)	0.12.3	generación y validación JWT
Backend	Spring Data JPA	3.x	Acceso a datos / ORM
Backend	Hibernate	6.x	implementación JPA
Backend	Lombok	1.18.38	reducción de boilerplate Java
Backend	Maven	3.8+	Build y dependencias
Frontend	React	19.2.4	Biblioteca de UI
Frontend	Vite	8.0.4	Bundler y dev server
Frontend	React Router DOM	7.14.0	Enrutamiento SPA
Frontend	Axios	1.15.0	Cliente HTTP con interceptores
Frontend	Recharts	3.8.1	gráficos de historial de precios
Frontend	Tailwind CSS	4.2.2	Utilidades CSS
Base de datos	MySQL	8.x	RDBMS principal

DieselMaps, extraído de universidad piloto de Colombia

Firmado por Líder de calidad

Ver 2.0

	MANUAL TECNICO	
Universidad Piloto de Colombia	PROYECTO: DieselMaps	Grupo: 2
		Ciclo: 2

Capa	Tecnologia	Version	Proposito
Base de datos	Flyway	incluido	Migraciones versionadas de esquema
Testing	JUnit 5 + Mockito	incluido	Pruebas unitarias backend

2.4 Capas del backend

Capa	Paquete	Responsabilidad
presentación	com.dieselmaps.controller	Recibe HTTP, valida entrada con @Valid, delega a Service, retorna ResponseEntity
Negocio	com.dieselmaps.service	lógica de negocio, orquestación de repositorios, transacciones @Transactional
Persistencia	com.dieselmaps.repository	Interfaces Spring Data JPA con queries personalizadas (JPQL y native)
Dominio	com.dieselmaps.entity	Entidades JPA que mapean las 11 tablas de MySQL con anotaciones de ciclo de vida
Transporte	com.dieselmaps.dto	19 DTOs de request/response que aíslan la API de las entidades internas
Seguridad	com.dieselmaps.security	JwtUtil, JwtFilter, UserDetailsServiceImpl
configuración	com.dieselmaps.config	SecurityConfig, DataInitializer, DataLoader
Errores	com.dieselmaps.exception	GlobalExceptionHandler (@RestControllerAdvice)

3. Estructura del Proyecto

3.1 árbol de directorios

DieselMaps, extraído de universidad piloto de Colombia

Firmado por líder de calidad

Ver 2.0

	MANUAL TECNICO	
Universidad Piloto de Colombia	PROYECTO: DieselMaps	Grupo: 2
		Ciclo: 2

```

DieselMapsFinal-main/
|-- backend/                # Aplicacion Spring Boot
|   |-- pom.xml             # Dependencias Maven
|   |-- src/
|       |-- main/java/com/dieselmaps/
|           |-- BackendApplication.java    # Punto de entrada
|           |-- config/                   # SecurityConfig, DataInitializer, DataLoader
|           |-- controller/               # 6 REST Controllers
|           |-- dto/                      # 19 DTOs de request/response
|           |-- entity/                   # 11 entidades JPA
|           |-- exception/                # GlobalExceptionHandler
|           |-- repository/               # 11 interfaces Spring Data
|           |-- security/                 # JwtUtil, JwtFilter
|           |-- service/                  # 6 servicios de negocio
|       |-- main/resources/
|           |-- application.properties    # Config principal
|           |-- application-dev.properties # Config desarrollo
|           |-- db/migration/             # V1.0.4 a V1.0.7 (Flyway)
|       |-- test/java/com/dieselmaps/     # 7 clases de prueba JUnit5
|-- frontend/                  # Aplicacion React + Vite
|   |-- package.json
|   |-- vite.config.js
|   |-- src/
|       |-- App.jsx            # Enrutamiento principal
|       |-- main.jsx           # Entry point React
|       |-- api/               # auth.js, fuel.js (clientes Axios)
|       |-- components/        # MapView, Navbar, ProtectedRoute,
|                               # StationCard, PriceHistoryChart, PriceTag
|       |-- constants/         # fuels.js (tipos de combustible)
|       |-- context/           # AuthContext.jsx
|       |-- hooks/             # useGeolocation.js
|       |-- pages/             # 12 vistas completas
|       |-- styles/            # store.css, favorites.css
|-- CHANGELOG.md
|-- GUIA_INICIO_RAPIDO.md

```

3.2 descripción de módulos clave

DieselMaps, extraído de universidad piloto de Colombia

Firmado por Líder de calidad

Ver 2.0

	MANUAL TECNICO	
Universidad Piloto de Colombia	PROYECTO: DieselMaps	Grupo: 2
		Ciclo: 2

Archivo / Modulo	descripción
SecurityConfig.java	Define la cadena de filtros: endpoints públicos, CORS, CSRF desactivado, STATELESS, agrega JwtFilter antes de UsernamePasswordAuthenticationFilter.
JwtUtil.java	Genera tokens HS256 con claims {sub, role, iat, exp}. validateToken() captura JwtException para rechazar tokens inválidos o expirados.
JwtFilter.java	OncePerRequestFilter: extrae Bearer token del header Authorization, valida y puebla SecurityContext antes de que el request llegue al Controller.
GlobalExceptionHandler.java	@RestControllerAdvice: MethodArgumentNotValidException -> 400 con mapa de errores; RuntimeException -> 400 con mensaje; sin stacktrace en respuesta.
FuellInventory.java	Entidad con lógica de negocio embebida: canPurchase(), completeImmediatePurchase(), reserveStock(), restock(). Constraint UNIQUE (station_id, fuel_type).
AuthContext.jsx	Estado global de autenticación: valida JWT al inicio, expone login() y logout(). Usa useMemo para evitar renders innecesarios del contexto.
ProtectedRoute.jsx	Guarda de ruta React: si user redirige a /login; si rol insuficiente redirige a /; soporta prop requiredRoles para control granular.
api/auth.js	Axios instance con interceptor de request (inyecta JWT) e interceptor de response (redirige a /login en 401 automáticamente).
useGeolocation.js	Hook: watchPosition con enableHighAccuracy, timeout 12s, maximumAge 60s. Limpia el watcher con clearWatch en el cleanup del useEffect.

4. configuración del Entorno

4.1 Requisitos previos

DieselMaps, extraído de universidad piloto de Colombia

Firmado por Líder de calidad

Ver 2.0

	MANUAL TECNICO	
Universidad Piloto de Colombia	PROYECTO: DieselMaps	Grupo: 2
		Ciclo: 2

Herramienta	versión mínima	propósito
JDK (Java Development Kit)	17 LTS	Compilar y ejecutar el backend Spring Boot
Apache Maven	3.8+	Build y gestión de dependencias del backend
Node.js	18 LTS+	Entorno de ejecución del frontend
npm	9+	Gestor de paquetes frontend (incluido con Node.js)
MySQL Server	8.0+	Base de datos relacional
Git	2.x	Control de versiones

4.2 instalación paso a paso

Paso 1 — Crear la base de datos MySQL

Conectarse como root y ejecutar:

```
CREATE DATABASE diesel_maps
  CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;

CREATE USER 'dm_user'@'localhost' IDENTIFIED BY 'TuPasswordSeguro';
GRANT ALL PRIVILEGES ON diesel_maps.* TO 'dm_user'@'localhost';
FLUSH PRIVILEGES;

-- Las tablas son creadas automáticamente por Flyway al arrancar el backend
```

Paso 2 — Configurar el backend

Editar backend/src/main/resources/application.properties:

```
# Conexion a base de datos
spring.datasource.url=jdbc:mysql://localhost:3306/diesel_maps?allowPublicKeyRetrieval=true&useSSL=false&serverTimezone=America/Bogota
spring.datasource.username=dm_user
```

DieselMaps, extraído de universidad piloto de Colombia

Firmado por líder de calidad

Ver 2.0

	MANUAL TECNICO		
Universidad Piloto de Colombia	PROYECTO: DieselMaps	Grupo: 2	
		Ciclo:	2

```
spring.datasource.password=TuPasswordSeguro

# JWT — CAMBIAR en produccion (minimo 32 caracteres)
app.jwt.secret=diesel_maps_secret_key_muy_larga_y_segura_2024
app.jwt.expiration=86400000

# CORS — ajustar al dominio del frontend en produccion
app.cors.allowed-origins=http://localhost:3000

# JPA
spring.jpa.show-sql=true # Desactivar en produccion
spring.jpa.hibernate.ddl-auto=update
```

Paso 3 — Compilar y ejecutar el backend

```
cd backend
./mvnw clean install -DskipTests
./mvnw spring-boot:run

# El servidor inicia en http://localhost:8080
# Flyway ejecuta automaticamente las migraciones pendientes al arrancar
```

Paso 4 — Instalar y ejecutar el frontend

```
cd frontend
npm install
npm run dev

# La app inicia en http://localhost:5173
```

4.3 Variables de configuración

Propiedad	Valor por defecto	descripción
spring.datasource.url	jdbc:mysql://localhost:3306/diesel_maps...	URL de conexión MySQL con parámetros de zona horaria

DieselMaps, extraído de universidad piloto de Colombia

Firmado por líder de calidad

Ver 2.0

	MANUAL TECNICO	
Universidad Piloto de Colombia	PROYECTO: DieselMaps	Grupo: 2
		Ciclo: 2

Propiedad	Valor por defecto	descripción
spring.datasource.username	root	Usuario MySQL — usar usuario dedicado en producción
spring.datasource.password	0305	CAMBIAR en producción. Nunca versionar en Git.
app.jwt.secret	diesel_maps_secret_key...	Clave de firma JWT — mínimo 32 chars en producción
app.jwt.expiration	86400000	expiración del token en milisegundos (24 horas)
app.cors.allowed-origins	http://localhost:3000	Origen CORS permitido — cambiar al dominio del frontend
spring.jpa.show-sql	true	Loguear SQL generado — desactivar en producción
server.port	8080	Puerto del servidor Spring Boot embebido (Tomcat)

ADVERTENCIA Seguridad en producción

- Reemplazar app.jwt.secret con una clave aleatoria de al menos 32 caracteres (256 bits).
- Externalizar credenciales de BD a variables de entorno del SO o un gestor de secretos (HashiCorp Vault).
- Desactivar spring.jpa.show-sql, spring.jpa.format_sql y spring.devtools.restart.enabled.
- Restringir app.cors.allowed-origins al dominio real del frontend.

5. Base de Datos

DieselMaps, extraído de universidad piloto de Colombia

Firmado por líder de calidad

Ver 2.0

	MANUAL TECNICO	
Universidad Piloto de Colombia	PROYECTO: DieselMaps	Grupo: 2
		Ciclo: 2

5.1 Motor y migraciones

DieselMaps usa MySQL 8.x con el motor InnoDB y el conjunto de caracteres utf8mb4. El esquema es gestionado por Flyway, cuyos scripts residen en backend/src/main/resources/db/migration/ con el esquema de nombrado V{MAJOR}.{MINOR}.{PATCH}__Descripcion.sql. Flyway valida el checksum de cada script ejecutado y rechaza re-ejecuciones, garantizando trazabilidad completa del esquema.

Migraciones incluidas: V1.0.4 (FuelInventoryHistory), V1.0.5 (backfill inventario), V1.0.6 (backfill precios GAS), V1.0.7 (StationStatusHistory).

5.2 Tablas principales

Tabla	PK	descripción y columnas clave
users	id BIGINT AI	Usuarios. Columnas: username UNIQUE, email UNIQUE, password_hash (BCrypt), phone, birth_date, role ENUM(USER OPERATOR ADMIN), active BOOLEAN, created_at.
stations	id BIGINT AI	Estaciones. FK operator_id->users. Columnas: name, brand, latitude DECIMAL(10,7), longitude DECIMAL(10,7), address, available BOOLEAN, updated_at.
fuel_prices	id BIGINT AI	Precio actual por tipo. UK (station_id, fuel_type). Columnas: price_cop DECIMAL(10,2), prev_price_cop DECIMAL(10,2), fuel_type ENUM.
fuel_inventory	id BIGINT AI	Stock por tipo. UK (station_id, fuel_type). Columnas: stock_available, stock_reserved, stock_total (DECIMAL 12,2), updated_at.
fuel_purchases	id BIGINT AI	Cabecera de compra. FK user_id, station_id. Columnas: total_amount DECIMAL(12,2), status ENUM(PENDING COMPLETED CANCELLED), purchase_date.
fuel_purchase_details	id BIGINT AI	Lineas de compra. FK purchase_id. Columnas: fuel_type ENUM, quantity_liters, unit_price, subtotal (DECIMAL 12,2).

DieselMaps, extraído de universidad piloto de Colombia

Firmado por Líder de calidad

Ver 2.0

	MANUAL TECNICO	
Universidad Piloto de Colombia	PROYECTO: DieselMaps	Grupo: 2
		Ciclo: 2

Tabla	PK	descripción y columnas clave
user_favorites	id BIGINT AI	Toggle Favorito. FK user_id, station_id. Columna: created_at.
alerts	id BIGINT AI	Notificaciones. FK user_id. Columnas: message TEXT, is_read BOOLEAN, created_at.
price_history	id BIGINT AI	Historial inmutable de precios. FK station_id. Columnas: fuel_type, price_cop, recorded_at.
fuel_inventory_history	id BIGINT AI	Audit trail de stock. FK station_id. Columnas: fuel_type, change_type ENUM, quantity_delta, previous_stock, new_stock, reason, recorded_at. Indices en (station_id, recorded_at) y (fuel_type, recorded_at).
station_status_history	id BIGINT AI	Historial de activacion/desactivacion. FK station_id. Columnas: previous_status, new_status, changed_by, reason, changed_at.

5.3 Diagrama de relaciones

```

users (1) ----- (N) stations      [FK operator_id]
stations (1) ----- (N) fuel_prices  UK(station_id, fuel_type)
stations (1) ----- (N) fuel_inventory UK(station_id, fuel_type)
stations (1) ----- (N) fuel_purchases [FK station_id]
fuel_purchases (1) ---- (N) fuel_purchase_details [FK purchase_id]
users (1) ----- (N) fuel_purchases [FK user_id]
users (1) ----- (N) user_favorites [FK user_id]
stations (1) ----- (N) user_favorites [FK station_id]
users (1) ----- (N) alerts          [FK user_id]
stations (1) ----- (N) price_history [FK station_id]
stations (1) ----- (N) fuel_inventory_history [FK station_id]
stations (1) ----- (N) station_status_history [FK station_id]

```

5.4 Ejemplo de registros

Tabla: users

DieselMaps, extraído de universidad piloto de Colombia

Firmado por Líder de calidad

Ver 2.0

	MANUAL TECNICO		
Universidad Piloto de Colombia	PROYECTO: DieselMaps	Grupo: 2	
		Ciclo:	2

```

id | username | email | role | active
---+-----+-----+-----+-----
1 | admin_root | admin@dieselmaps.com | ADMIN | true
2 | op_terpel | op@terpel.com | OPERATOR | true
3 | juan_user | juan@gmail.com | USER | true

```

Tabla: fuel_inventory

```

id | station_id | fuel_type | stock_available | stock_reserved | stock_total
---+-----+-----+-----+-----+-----
1 | 1 | DIESEL | 1000.00 | 50.00 | 1050.00
2 | 1 | CORRIENTE | 500.00 | 0.00 | 500.00
3 | 1 | EXTRA | 300.00 | 0.00 | 300.00
4 | 1 | GAS | 200.00 | 0.00 | 200.00

```

6. Backend

6.1 Referencia completa de endpoints API

Metodo	Endpoint	Auth requerida	descripción
POST	/api/auth/register	No	Registrar usuario (USER u OPERATOR). No permite rol ADMIN.
POST	/api/auth/login	No	Login con username/password; retorna JWT + rol.
GET	/api/stations/public/nearby?lat&lng&radiusKm	No	Estaciones cercanas — endpoint publico legacy.

DieselMaps, extraído de universidad piloto de Colombia

Firmado por Líder de calidad

Ver 2.0

	MANUAL TECNICO	
Universidad Piloto de Colombia	PROYECTO: DieselMaps	Grupo: 2
		Ciclo: 2

Metodo	Endpoint	Auth requerida	descripción
GET	/api/stations/nearby?lat&lng&radius	No	Estaciones cercanas — alias optimizado para UI moderna.
POST	/api/stations	OPERATOR / ADMIN	Crear nueva estacion con precios e inventario inicial.
GET	/api/stations/{id}	Autenticado	Detalle de estacion con precios e inventario.
PATCH	/api/stations/{id}/prices	OPERATOR / ADMIN	Actualizar precio; genera PriceHistory y Alerts.
GET	/api/stations/{id}/history	Autenticado	Historial cronologico de precios.
PATCH	/api/stations/{id}/inventory/{fuelType}	OPERATOR / ADMIN	Ajustar stock con delta positivo o negativo.
GET	/api/stations/{id}/inventory/history	Autenticado	Historial de cambios de inventario de la estacion.
GET	/api/fuel/inventory/station/{id}	Autenticado	Inventario completo de todos los tipos de combustible.
GET	/api/fuel/inventory/station/{id}/fuel/{ft}	Autenticado	Inventario de un tipo de combustible especifico.
PATCH	/api/fuel/inventory/station/{id}/fuel/{ft}/adjust	OPERATOR / ADMIN	Ajuste con delta (+/-).
PATCH	/api/fuel/inventory/station/{id}/fuel/{ft}/restock	OPERATOR / ADMIN	Reabastecimiento (suma litros al stock).
PUT	/api/fuel/inventory/station/{id}/fuel/{ft}	OPERATOR / ADMIN	Establecer stock absoluto.
GET	/api/fuel/inventory/station/{id}/history	Autenticado	Historial de cambios de inventario.
POST	/api/fuel/purchases	USER / OPERATOR / ADMIN	Crear compra virtual atomica con descuento de stock.

DieselMaps, extraído de universidad piloto de Colombia

Firmado por Líder de calidad

Ver 2.0

	MANUAL TECNICO	
Universidad Piloto de Colombia	PROYECTO: DieselMaps	Grupo: 2
		Ciclo: 2

Metodo	Endpoint	Auth requerida	descripción
GET	/api/fuel/purchases/my	USER / OPERATOR / ADMIN	Historial de compras del usuario autenticado.
GET	/api/fuel/purchases/{id}	USER / OPERATOR / ADMIN	Detalle de una compra específica.
GET	/api/fuel/purchases/station/{id}	OPERATOR / ADMIN	Historial de compras de una estacion.
POST	/api/users/favorites/{stationId}	Autenticado	Toggle favorito — agrega si no existe, elimina si ya existe.
GET	/api/users/favorites	Autenticado	Listar estaciones favoritas del usuario.
GET	/api/users/alerts	Autenticado	Listar alertas de cambio de precio.
PUT	/api/users/alerts/{alertId}/read	Autenticado	Marcar alerta como leída.
GET	/api/admin/stations	ADMIN	Listar todas las estaciones del sistema.
PUT	/api/admin/stations/{id}	ADMIN	Actualizar datos de una estacion.
DELETE	/api/admin/stations/{id}	ADMIN	Desactivar estacion (soft delete).
PUT	/api/admin/stations/{id}/activate	ADMIN	Activar estacion.
PUT	/api/admin/stations/{id}/deactivate	ADMIN	Desactivar estacion.
PUT	/api/admin/stations/{id}/prices	ADMIN	Actualizar precios en bloque.
PUT	/api/admin/stations/{id}/inventory	ADMIN	Actualizar inventario en bloque.

6.2 Ejemplos de request y response

DieselMaps, extraído de universidad piloto de Colombia

Firmado por Líder de calidad

Ver 2.0

	MANUAL TECNICO		
Universidad Piloto de Colombia	PROYECTO: DieselMaps	Grupo: 2	
		Ciclo:	2

POST /api/auth/login

Request body:

```
{
  "username": "juan_user",
  "password": "MiPassword123"
}
```

Response 200 OK:

```
{
  "token": "eyJhbGciOiJIUzI1NiJ9.eyJzdWl0OiJqdWFnX3VzZXli...",
  "username": "juan_user",
  "role": "USER"
}
```

POST /api/fuel/purchases

Request body:

```
{
  "stationId": 1,
  "items": [
    { "fuelType": "DIESEL", "quantityLiters": 50.00 },
    { "fuelType": "CORRIENTE", "quantityLiters": 30.00 }
  ]
}
```

Response 201 Created:

```
{
  "id": 5, "userId": 3, "username": "juan_user",
  "stationId": 1, "stationName": "Estacion Central",
  "totalAmount": 632000.00, "status": "COMPLETED",
  "purchaseDate": "2026-05-01T10:30:00",
  "details": [
    { "fuelType": "DIESEL", "quantityLiters": 50.0, "unitPrice": 9800.0, "subtotal": 490000.0 },
    { "fuelType": "CORRIENTE", "quantityLiters": 30.0, "unitPrice": 4700.0, "subtotal": 141000.0 }
  ]
}
```

DieselMaps, extraído de universidad piloto de Colombia

Firmado por líder de calidad

Ver 2.0

	MANUAL TECNICO		
Universidad Piloto de Colombia	PROYECTO: DieselMaps	Grupo: 2	
		Ciclo:	2

6.3 lógica de negocio critica

Compra atomica — FuelPurchaseService.createPurchase()

Anotada con `@Transactional(isolation = Isolation.READ_COMMITTED)` para evitar race conditions en el stock. El flujo es:

1. Validar que usuario y estación existan y que la estación este activa (`available = true`).
2. Validar que la lista de items no este vacía y que cada `quantityLiters` sea > 0 .
3. Por cada item: obtener inventario con `findByStationIdAndFuelTypeForUpdate()` (bloqueo pesimista), llamar `canPurchase()`, obtener precio actual.
4. Calcular `totalAmount = suma (quantity * priceCop)` para cada tipo.
5. Persistir FuelPurchase con estatus = COMPLETED y sus FuelPurchaseDetail.
6. Para cada item: llamar `completeImmediatePurchase()` en FuelInventory y persistir; registrar FuelInventoryHistory.

Un fallo en cualquier paso provoca rollback completo, garantizando la integridad del stock.

actualización de precio con historial y alertas — StationService.updatePrice()

7. Buscar FuelPrice por (`stationId, fuelType`); lanzar excepcion si no existe.
8. Guardar `prevPriceCop` y actualizar `priceCop` al nuevo valor.
9. Crear y persistir registro en PriceHistory.
10. Buscar todos los UserFavorite de la estacion con `userFavoriteRepository.findByStationId()`.
11. Por cada usuario favorito, crear Alert con el mensaje de cambio de precio.
12. Persistir todas las alertas con `alertRepository.saveAll()`.

7. Frontend

7.1 organización de archivos

DieselMaps, extraído de universidad piloto de Colombia

Firmado por líder de calidad

Ver 2.0

	MANUAL TECNICO	
Universidad Piloto de Colombia	PROYECTO: DieselMaps	Grupo: 2
		Ciclo: 2

Archivo / Directorio	descripción
src/App.jsx	Configura BrowserRouter + AuthProvider + Routes con React Router v7. Define que rutas requieren ProtectedRoute y que roles son necesarios.
src/main.jsx	Entry point: monta React.StrictMode + App en el nodo #root del index.html.
src/context/AuthContext.jsx	Estado global de autenticación. Persiste token y user en localStorage. Valida expiracion JWT al montar la app. Expone login() y logout() a través del hook useAuth().
src/api/auth.js	Instancia Axios con baseURL http://localhost:8080/api. Interceptor de request inyecta JWT. Interceptor de response redirige a /login en 401. Exporta: authAPI, stationsAPI, adminStationsAPI, usersAPI.
src/api/fuel.js	Cientes para fuelInventoryAPI y fuelPurchaseAPI con endpoints /api/fuel/*.
src/hooks/useGeolocation.js	Hook personalizado: watchPosition con enableHighAccuracy, timeout 12s, maximumAge 60s. Retorna { location, loading, error }.
src/components/ProtectedRoute.jsx	Guarda de ruta: si !user -> /login; si requiredRoles y rol insuficiente -> /; si loading -> spinner.
src/components/MapView.jsx	Mapa interactivo con @react-google-maps/api. Muestra marcadores de estaciones con informacion de precio e inventario.
src/components/Navbar.jsx	Barra de navegacion adaptativa: muestra opciones segun el rol del usuario (USER, OPERATOR, ADMIN).
src/components/PriceHistoryChart.jsx	Grafico de lineas Recharts del historial de precios de una estacion.
src/constants/fuels.js	Enumeracion de tipos de combustible: CORRIENTE, EXTRA, DIESEL, GAS con labels en espanol y colores.
src/pages/	12 vistas completas: Login, Register, Home, Map, StationDetail, CreateStation, Store, Favorites, Routes, Compare, AdminStations, NotFound.

7.2 Rutas y control de acceso

DieselMaps, extraído de universidad piloto de Colombia

Firmado por Líder de calidad

Ver 2.0

	MANUAL TECNICO		
Universidad Piloto de Colombia	PROYECTO: DieselMaps	Grupo: 2	
		Ciclo:	2

Ruta	Componente	Auth	Roles permitidos
/	Home	No	Todos
/login	Login	No	Todos
/register	Register	No	Todos
/map	Map	Si	Todos los autenticados
/station/:id	StationDetail	Si	Todos los autenticados
/create-station	CreateStation	Si	OPERATOR, ADMIN
/favorites	Favorites	Si	Todos los autenticados
/routes	RoutesPage	Si	Todos los autenticados
/compare	Compare	Si	Todos los autenticados
/store	Store	Si	Todos los autenticados
/admin/stations	AdminStations	Si	ADMIN solamente
*	NotFound	No	Todos

7.3 gestión del estado de autenticación

AuthContext implementa el patron Context + Provider. Al montar la app parsea el token JWT almacenado en localStorage, verifica que no haya expirado comparando decoded.exp con Date.now() / 1000, y solo inicializa el estado user si es valido. En caso contrario, limpia el localStorage y arranca con user = null.

```
// Claves usadas en localStorage
const TOKEN_KEY = 'dieselmaps_token';
const USER_KEY = 'dieselmaps_user';

// login(): almacena token y user, navega a /map
const login = ({ username, role }, token) => {
  localStorage.setItem(TOKEN_KEY, token);
  localStorage.setItem(USER_KEY, JSON.stringify({ username, role }));
  setUser({ username, role });
}
```

DieselMaps, extraído de universidad piloto de Colombia

Firmado por líder de calidad

Ver 2.0

	MANUAL TECNICO		
Universidad Piloto de Colombia	PROYECTO: DieselMaps	Grupo: 2	
		Ciclo:	2

```

    navigate('/map');
  };

  // logout(): limpia localStorage, user=null, navega a /login
  const logout = () => {
    localStorage.removeItem(TOKEN_KEY);
    localStorage.removeItem(USER_KEY);
    setUser(null);
    navigate('/login');
  };

```

7.4 Interceptores Axios

```

// Interceptor de REQUEST: inyecta JWT en cada peticion
API.interceptors.request.use((config) => {
  const token = localStorage.getItem('dieselmaps_token');
  if (token) config.headers.Authorization = `Bearer ${token}`;
  return config;
});

// Interceptor de RESPONSE: logout automatico en 401
API.interceptors.response.use(
  (response) => response,
  (error) => {
    if (error.response?.status === 401) {
      localStorage.removeItem('dieselmaps_token');
      localStorage.removeItem('dieselmaps_user');
      window.location.href = '/login';
    }
    return Promise.reject(error);
  }
);

```

8. Seguridad

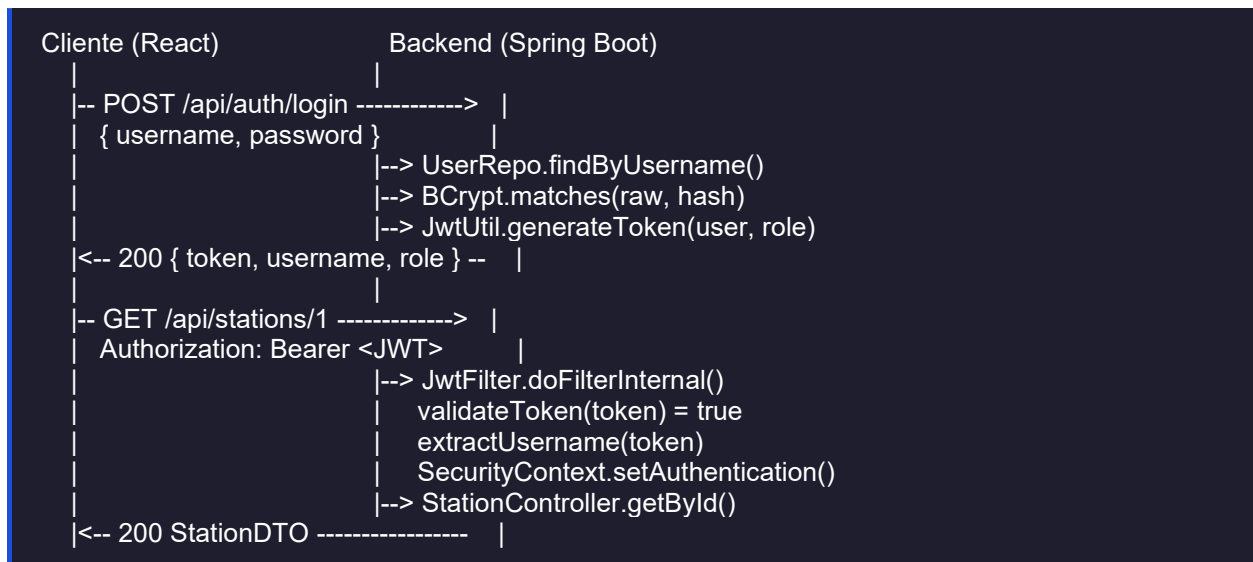
DieselMaps, extraído de universidad piloto de Colombia

Firmado por Líder de calidad

Ver 2.0

	MANUAL TECNICO		
Universidad Piloto de Colombia	PROYECTO: DieselMaps	Grupo: 2	
		Ciclo:	2

8.1 Flujo completo de autenticación



8.2 JWT — implementación

- **Algoritmo:** HS256 (HMAC-SHA256) con clave configurable en app.jwt.secret.
- **Claims incluidos:** sub (username), role (USER/OPERATOR/ADMIN), iat (emision), exp (expiracion).
- **Expiracion:** configurable via app.jwt.expiration (defecto: 86400000 ms = 24 horas).
- **Almacenamiento en cliente:** localStorage['dieselmaps_token'].
- **Validacion:** JwtUtil.validateToken() captura JwtException e IllegalArgumentException. Cualquier manipulacion de la firma invalida el token.

```

// JwtUtil.generateToken()
return Jwts.builder()
    .setSubject(username)
    .claim("role", role)
    .setIssuedAt(new Date())
    .setExpiration(new Date(System.currentTimeMillis() + expiration))
    .signWith(key, SignatureAlgorithm.HS256)
    .compact();
  
```

DieselMaps, extraído de universidad piloto de Colombia

Firmado por líder de calidad

Ver 2.0

	MANUAL TECNICO	
Universidad Piloto de Colombia	PROYECTO: DieselMaps	Grupo: 2 Ciclo: 2

8.3 autorización RBAC

Recurso	Nivel de acceso	Mecanismo
/api/auth/**	Publico (sin auth)	SecurityConfig: .requestMatchers('/api/auth/**').permitAll()
/api/stations/public/**	Publico	SecurityConfig: .requestMatchers('/api/stations/public/**').permitAll()
/api/admin/**	ADMIN solamente	SecurityConfig: .hasRole('ADMIN') + @PreAuthorize a nivel de clase en AdminStationController
/api/stations (POST)	OPERATOR o ADMIN	@PreAuthorize("hasAnyRole('OPERATOR', 'ADMIN')")
/api/fuel/purchases (POST)	USER / OPERATOR / ADMIN	@PreAuthorize("hasAnyRole('USER', 'OPERATOR', 'ADMIN')")
/api/users/**	Cualquier autenticado	anyRequest().authenticated() en SecurityConfig

8.4 Manejo de contraseñas

- BCryptPasswordEncoder con factor de costo por defecto (10 rondas).
- La entidad User almacena passwordHash: nunca el texto plano.
- En login: passwordEncoder.matches(rawPassword, hash) — nunca se descifra el hash.
- El registro valida explícitamente que role != 'ADMIN'.

8.5 configuración CORS

```
// SecurityConfig.corsConfigurationSource()
configuration.setAllowedOriginPatterns(Arrays.asList("http://localhost:*"));
configuration.setAllowedMethods(Arrays.asList("GET","POST","PUT","PATCH","DELETE","OPTIONS"));
```

DieselMaps, extraído de universidad piloto de Colombia

Firmado por líder de calidad

Ver 2.0

	MANUAL TECNICO	
Universidad Piloto de Colombia	PROYECTO: DieselMaps	Grupo: 2
		Ciclo: 2

```
configuration.setAllowedHeaders(Arrays.asList("*"));
configuration.setAllowCredentials(true);

// En produccion: reemplazar localhost:* por el dominio real del frontend
// configuration.setAllowedOriginPatterns(List.of("https://dieselmaps.ejemplo.com"));
```

8.6 Validaciones de entrada

Todos los endpoints que reciben body usan `@Valid` junto a anotaciones JSR-303 en los DTOs. Ejemplos reales del proyecto:

```
// RegisterRequest.java
@NotBlank(message = "El username es obligatorio")
private String username;

@email(message = "Email invalido")
@NotBlank
private String email;

@Size(min = 6, message = "La contraseña debe tener al menos 6 caracteres")
private String password;
```

9. Manejo de Errores y Logs

9.1 GlobalExceptionHandler

El `@RestControllerAdvice` centraliza el manejo de excepciones y garantiza respuestas JSON consistentes. ningún stacktrace se expone al cliente en producción.

DieselMaps, extraído de universidad piloto de Colombia

Firmado por líder de calidad

Ver 2.0

	MANUAL TECNICO	
Universidad Piloto de Colombia	PROYECTO: DieselMaps	Grupo: 2
		Ciclo: 2

excepción capturada	HTTP Status	Formato de respuesta
MethodArgumentNotValidException	400 Bad Request	{ "campo1": "mensaje", "campo2": "mensaje" }
RuntimeException	400 Bad Request	{ "error": "mensaje", "message": "mensaje" }
IllegalArgumentException	400 Bad Request	{ "error": "mensaje", "message": "mensaje" }
JWT invalido / ausente	401 Unauthorized	Respuesta vacia de Spring Security
Rol insuficiente	403 Forbidden	Respuesta vacia de Spring Security

9.2 Errores de negocio frecuentes

Mensaje de error	Causa en el codigo	Servicio
El correo ya esta registrado	userRepository.existsByEmail() = true	AuthService.register()
No puedes registrarte como ADMIN	role = 'ADMIN' en RegisterRequest	AuthService.register()
Credenciales incorrectas	findByUsername() empty o matches() false	AuthService.login()
Estacion no encontrada	findById() retorna Optional.empty()	StationService / AdminStationService
La estacion no esta disponible	station.isAvailable() = false al comprar	FuelPurchaseService.createPurchase()
Stock insuficiente de {tipo}	inventory.canPurchase(quantity) = false	FuelPurchaseService / FuelInventory
La cantidad debe ser mayor a 0	quantityLiters <= 0 en item de compra	FuelPurchaseService.createPurchase()
No tienes permiso	Alerta pertenece a diferente usuario	UserService.markAlertAsRead()

DieselMaps, extraído de universidad piloto de Colombia

Firmado por líder de calidad

Ver 2.0

	MANUAL TECNICO		
Universidad Piloto de Colombia	PROYECTO: DieselMaps	Grupo: 2	
		Ciclo:	2

9.3 configuración de logging

En desarrollo (application-dev.properties):

```
logging.level.org.springframework=DEBUG
logging.level.com.dieselmaps=DEBUG
logging.level.org.hibernate.SQL=DEBUG
logging.level.org.hibernate.type.descriptor.sql.BasicBinder=TRACE
```

En produccion (desactivar SQL y reducir nivel):

```
logging.level.root=WARN
logging.level.com.dieselmaps=INFO
logging.file.name=/var/log/dieselmaps/app.log
logging.logback.rollingpolicy.max-file-size=10MB
logging.logback.rollingpolicy.max-history=30
```

10. Pruebas

10.1 Estructura de pruebas del backend

Clase de prueba	Clase bajo prueba	Cobertura
AuthServiceTest.java	AuthService	11 tests: registro (duplicado email, rol ADMIN, hash BCrypt, rol invalido), login (usuario invalido, password incorrecta, exito), mensaje de error homogéneo.
JwtUtilTest.java	JwtUtil	generación de token con claims, extracción de username, validación token valido, rechazo de token expirado y manipulado.

DieselMaps, extraído de universidad piloto de Colombia

Firmado por líder de calidad

Ver 2.0

	MANUAL TECNICO		
Universidad Piloto de Colombia	PROYECTO: DieselMaps	Grupo: 2	
		Ciclo:	2

Clase de prueba	Clase bajo prueba	Cobertura
AuthControllerTest.java	AuthController	Capa HTTP con @WebMvcTest: códigos HTTP correctos, binding de DTOs, manejo de errores de validación.
GlobalExceptionHandlerTest.java	GlobalExceptionHandler	Respuestas de error estructuradas: formato JSON correcto, ausencia de stacktrace.
StationServiceTest.java	StationService	CRUD de estaciones, búsqueda geoespacial, actualización de precio con historial y alertas.
UserServiceTest.java	UserService	Toggle favoritos (agregar y eliminar), listado de favoritos, marcado de alertas como leídas.
BackendApplicationTests.java	BackendApplication	Smoke test: verifica que el contexto Spring carga correctamente (@SpringBootTest).

10.2 Ejemplo de prueba unitaria con Mockito

```
@ExtendWith(MockitoExtension.class)
class AuthServiceTest {
    @Mock UserRepository userRepository;
    @Mock BCryptPasswordEncoder passwordEncoder;
    @Mock JwtUtil jwtUtil;
    @InjectMocks AuthService authService;

    @Test
    @DisplayName("Password se almacena como hash BCrypt, nunca en texto plano")
    void register_PasswordIsHashed() {
        when(userRepository.existsByEmail(anyString())).thenReturn(false);
        when(passwordEncoder.encode("Password1")).thenReturn("$2a$10$hashed");
        when(userRepository.save(any(User.class))).thenAnswer(inv -> {
            User u = inv.getArgument(0);
            assertThat(u.getPasswordHash()).isNotEqualTo("Password1"); // nunca plano
            assertThat(u.getPasswordHash()).startsWith("$2a$"); // BCrypt
            return savedUser;
        });
    }
}
```

DieselMaps, extraído de universidad piloto de Colombia

Firmado por líder de calidad

Ver 2.0

	MANUAL TECNICO		
Universidad Piloto de Colombia	PROYECTO: DieselMaps	Grupo: 2	
		Ciclo:	2

```
});
when(jwtUtil.generateToken(any(), any()))thenReturn("token");
authService.register(validRegisterRequest);
verify(passwordEncoder).encode("Password1");
}
}
```

10.3 Como ejecutar las pruebas

Backend — JUnit 5 + Mockito

```
# Ejecutar todas las pruebas
cd backend && ./mvnw test

# Ejecutar una clase especifica
./mvnw test -Dtest=AuthServiceTest

# Ejecutar con reporte de cobertura JaCoCo
./mvnw test jacoco:report
# Reporte HTML: backend/target/site/jacoco/index.html
```

Frontend — Lint

```
cd frontend
npm run lint # Analisis estatico ESLint
```

11. Despliegue

11.1 Build del backend

```
cd backend
```

DieselMaps, extraído de universidad piloto de Colombia

Firmado por líder de calidad

Ver 2.0

	MANUAL TECNICO		
Universidad Piloto de Colombia	PROYECTO: DieselMaps	Grupo: 2	
		Ciclo:	2

```
./mvnw clean package -DskipTests
# Genera: backend/target/backend-0.0.1-SNAPSHOT.jar
# El JAR incluye Tomcat embebido — no requiere servidor externo
```

11.2 Ejecutar el JAR en producción

```
java -jar backend/target/backend-0.0.1-SNAPSHOT.jar \
--spring.datasource.url=jdbc:mysql://prod-host:3306/diesel_maps \
--spring.datasource.username=dm_user \
--spring.datasource.password=ContraseniaSegura \
--app.jwt.secret=ClaveJWTAleatoria256bitsMinimo \
--app.cors.allowed-origins=https://dieselmaps.ejemplo.com
```

11.3 Build del frontend

```
cd frontend
npm run build
# Genera: frontend/dist/ (archivos estaticos para servir con Nginx o CDN)
```

11.4 configuración de Nginx

```
# /etc/nginx/sites-available/dieselmaps
server {
    listen 80;
    server_name dieselmaps.ejemplo.com;

    # Frontend React (SPA) — archivos estaticos
    root /var/www/dieselmaps/dist;
    index index.html;
    try_files $uri $uri/ /index.html; # Critico para React Router

    # Proxy inverso al backend Spring Boot
```

DieselMaps, extraído de universidad piloto de Colombia

Firmado por líder de calidad

Ver 2.0

	MANUAL TECNICO	
Universidad Piloto de Colombia	PROYECTO: DieselMaps	Grupo: 2
		Ciclo: 2

```
location /api/ {
    proxy_pass http://localhost:8080;
    proxy_set_header Host      $host;
    proxy_set_header X-Real-IP  $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
}
}
```

11.5 Servicio systemd (Linux)

```
# /etc/systemd/system/dieselmaps.service
[Unit]
Description=DieselMaps Backend
After=network.target mysql.service

[Service]
User=dieselmaps
WorkingDirectory=/opt/dieselmaps
ExecStart=/usr/bin/java -jar /opt/dieselmaps/backend.jar
EnvironmentFile=/opt/dieselmaps/.env
Restart=always
RestartSec=10

[Install]
WantedBy=multi-user.target

# Comandos de gestion
sudo systemctl daemon-reload
sudo systemctl enable --now dieselmaps
sudo journalctl -u dieselmaps -f # Ver logs en tiempo real
```

CHECKLIST de despliegue Verificar antes de publicar en producción

- Cambiar app.jwt.secret por clave aleatoria >= 32 caracteres (256 bits).
- Actualizar app.cors.allowed-origins al dominio real del frontend.
- Desactivar spring.jpa.show-sql, spring.jpa.format_sql y spring.devtools.
- Configurar HTTPS en Nginx con certificado SSL (Let's Encrypt recomendado).
- Habilitar backup automatico de MySQL (mysqldump diario).

DieselMaps, extraído de universidad piloto de Colombia

Firmado por líder de calidad

Ver 2.0

	MANUAL TECNICO	
Universidad Piloto de Colombia	PROYECTO: DieselMaps	Grupo: 2
		Ciclo: 2

- Verificar que Flyway ejecuto todas las migraciones: revisar tabla flyway_schema_history.
- Probar el endpoint /api/auth/login con las credenciales de prueba en el nuevo entorno.

13. Mantenimiento

12.1 Agregar una nueva migración de base de datos

14. Crear el archivo en backend/src/main/resources/db/migration/:

```
V{MAJOR}.{MINOR}.{PATCH}__Descripcion_del_cambio.sql
```

15. Escriba SQL idempotent usando IF NOT EXISTS / IF EXISTS en ALTER TABLE.
16. NUNCA modificar un script ya ejecutado: Flyway verifica el checksum y lanzara una excepcion al arrancar.
17. Al reiniciar el backend, Flyway detecta y ejecuta el nuevo script automaticamente.
18. Documentar el script de rollback (DROP, ALTER TABLE REVERT) junto a la migracion para casos de emergencia.

12.2 Agregar un nuevo endpoint

19. Implementar el método en el Service correspondiente con la logica de negocio y @Transactional.
20. Agregar el metodo al Controller con la anotacion @PreAuthorize correcta segun el rol minimo requerido.
21. Crear los DTOs de request/response si son necesarios.
22. Escribir prueba unitaria en el directorio test/ cubriendo flujos principal y casos borde.
23. Actualizar la funcion correspondiente en src/api/ del frontend.

12.3 Actualizar dependencias

DieselMaps, extraído de universidad piloto de Colombia

Firmado por líder de calidad

Ver 2.0

	MANUAL TECNICO	
Universidad Piloto de Colombia	PROYECTO: DieselMaps	Grupo: 2
		Ciclo: 2

```
# Backend: verificar versiones disponibles
./mvnw versions:display-dependency-updates
```

```
# Frontend: verificar actualizaciones
npm outdated
npm update
```

12.4 Buenas prácticas de mantenimiento

- **Commits semánticos:** feat(modulo): descripcion / fix(modulo): descripcion / docs: / refactor: / test:
- **Ramas de trabajo:** feature/nombre-funcionalidad para nuevas features; hotfix/descripcion para correcciones urgentes. Nunca trabajar directamente en main.
- **Code Review obligatorio:** Todo Pull Request requiere aprobacion de al menos un desarrollador antes del merge a develop o main.
- **Tests antes de merge:** El pipeline CI debe ejecutar todas las pruebas en verde antes de permitir el merge.
- **Backups de BD:** Programar mysqldump diario con retencion de 30 dias. Verificar la integridad del backup mensualmente.
- **CHANGELOG.md:** Registrar cada release siguiendo Semantic Versioning (MAJOR.MINOR.PATCH).

12.5 Rollback de emergencia

```
# Detener el servicio
sudo systemctl stop dieselmaps
```

```
# Restaurar el JAR anterior
cp /opt/dieselmaps/backup/backend-v0.9.jar /opt/dieselmaps/backend.jar
```

```
# Reiniciar el servicio
sudo systemctl start dieselmaps
```

```
# NOTA: si se ejecutaron migraciones de BD, el rollback de Flyway
# es manual. Ejecutar el script de rollback documentado junto a la migracion.
```

DieselMaps, extraído de universidad piloto de Colombia

Firmado por líder de calidad

Ver 2.0

	MANUAL TECNICO		
Universidad Piloto de Colombia	PROYECTO: DieselMaps	Grupo: 2	
		Ciclo:	2

Versión 2.0 — DieselMaps — Mayo 2026